COMPUTER ENGINEERING

QUEZON CITY

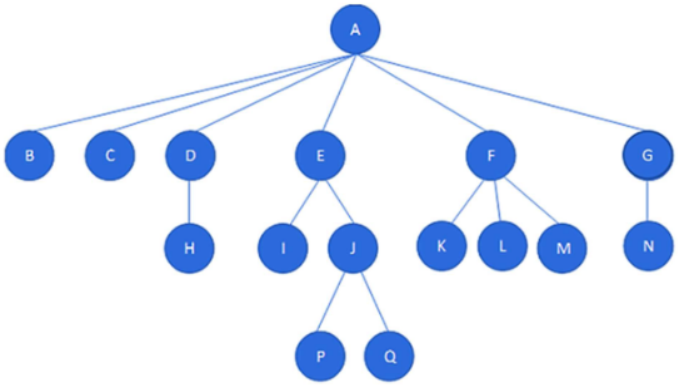
| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE010 - Data Structures & Algorithms | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 09/18/26 | **DATE SUBMITTED:** | 09/21/26 |
| **NAME:** | Arididdon, Gabriel Dwyane D | **INSTRUCTOR:** | Engr. Jimlord M. Quejado |

**ACTIVITY NO** **Hands-on Activity 9.1 Binary Search Tree**

## 1. PROCEDURE OUTPUT

**ILO A: Create C++ code to implement a general tree, binary tree and binary search tree**

**Task 1: Create code in C++ that will create a tree as shown in the figure above. Use linked lists as the internal representation of this tree. Indicate your code screenshot and comments in table 9-1.**




**Task 2: Complete the following table:**

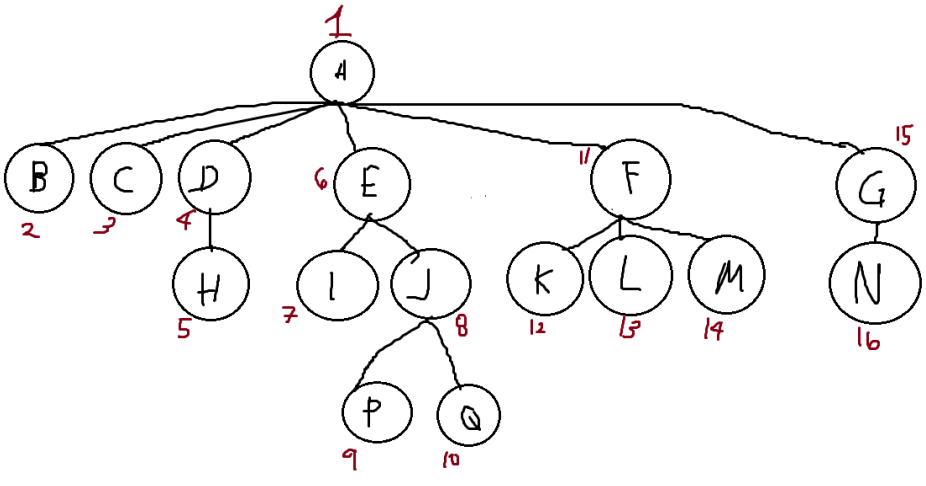
| Node | Height | Depth |
|---|---|---|
| A | 3 | 0 |
| B | 0 | 1 |
| C | 0 | 1 |
| D | 1 | 1 |
| E | 2 | 1 |
| F | 1 | 1 |
| G | 1 | 1 |
| H | 0 | 2 |

| | | |
|---|---|---|
| I | 0 | 2 |
| J | 1 | 2 |
| K | 0 | 2 |
| L | 0 | 2 |
| M | 0 | 2 |
| N | 0 | 2 |
| O | 0 | 2 |
| P | 0 | 3 |
| Q | 0 | 3 |

Table 9-2. Height and Depth of each Node

**ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.**

Task 3: After implementing the code for above, answer the following:
3.1 Given the tree diagram, find the result of the pre-order, post-order and in-order traversal strategies by hand. Include this as table 9-3 in section 6

| | |
|---|---|
| Pre-order |  |

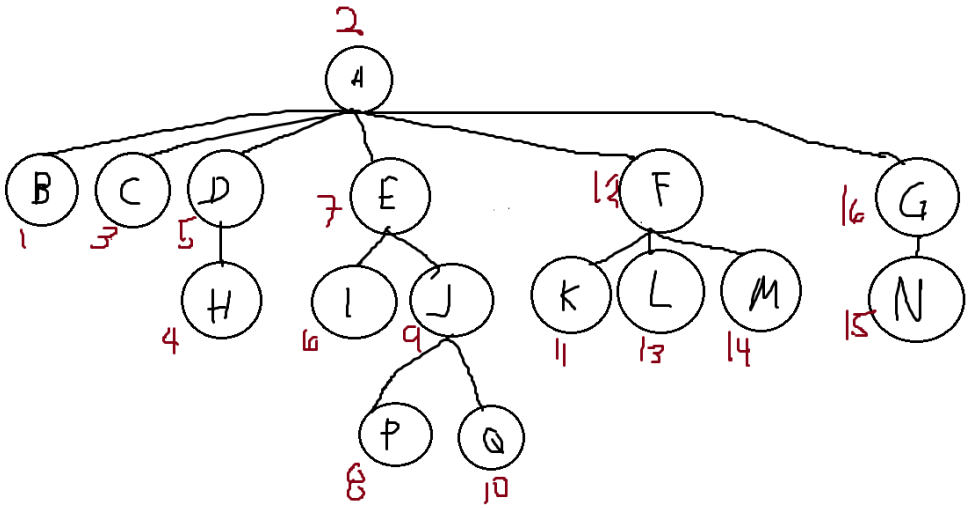
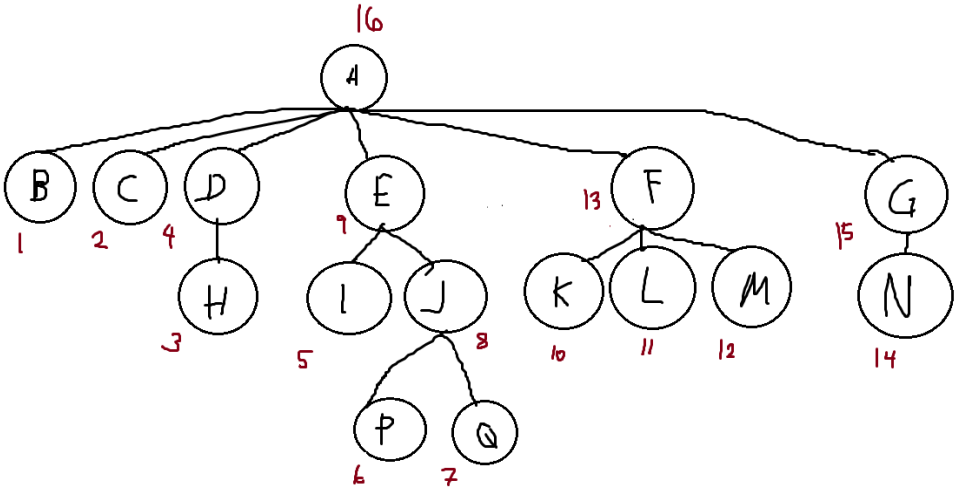
| | |
|---|---|
| In-order |  |
| Post-order |  |

Table 9-3. Pre-order, In-order, Post-order output by hand

3.2 Create a function for pre-order, post-order and in-order traversal. Make sure that each function displays an output into the console. Once you have the output, create and fill table 9-4 in section 6 so that it contains the screenshot of the function, screenshot of the output and your observations. Your observations consist of a comparison between output in #1 and the output of your functions in #2.

Pre-order

```cpp
template <typename T>
void GeneralTree<T>::preOrder() {
    std::cout << key << " ";
    GeneralTree<T>* child = firstChild;
    while (child != nullptr) {
        child->preOrder();
        child = child->nextSibling;
    }
}
```

In-order

```cpp
template <typename T>
void GeneralTree<T>::inOrder() {
    GeneralTree<T>* child = firstChild;
    if (child != nullptr) {
        child->inOrder();
        std::cout << key << " ";
        child = child->nextSibling;
        while (child != nullptr) {
            child->inOrder();
            child = child->nextSibling;
        }
    } else {
        std::cout << key << " ";
    }
}
```

Post-order

```cpp
template <typename T>
void GeneralTree<T>::postOrder() {
    GeneralTree<T>* child = firstChild;
    while (child != nullptr) {
        child->postOrder();
        child = child->nextSibling;
    }
    std::cout << key << " ";
}
```

Output

```
PREORDER:
A B C D H E I J P Q F K L M G N
INORDER:
B A C H D I E P J Q K F L M N G
POSTORDER:
B C H D I P Q J E K L M F N G A
```

| | |
|---|---|
| Observation | The expected output is accurate on the output on the terminal. The general tree's insert(parentValue, newValue) works by: find() searches the tree (recursively, via firstChild/nextSibling) for the node holding parentValue, then addChild() walks that node's sibling chain and appends the new node at the end. |

Table 9-4. Pre-order, In-order, Post-order output in terminal

3.3. Create a function called findData that will take the following parameter: “CHOICE, KEY”. Choice will determine which traversal method will be used to traverse the tree to find the second parameter KEY. If the KEY is found, the value will be displayed such that: “{KEY} was found!”. Otherwise, no output. Include your output as table 9-5 in section 6.

Code Screenshot

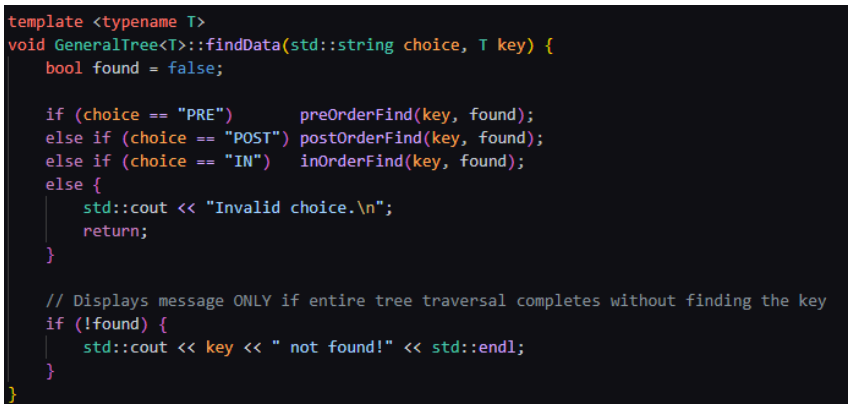
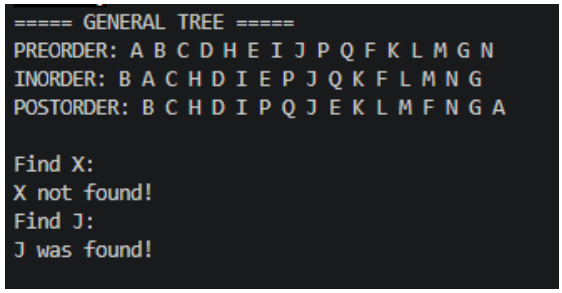
```
std::cout << "Find X:" << std::endl;
gTree.findData("PRE", 'X');

std::cout << "Find J:" << std::endl;
gTree.findData("PRE", 'J');
```

```
template <typename T>
void GeneralTree<T>::preOrderFind(T target, bool& found) {
    if (found) return;

    // 1. Visit current node
    if (key == target) {
        std::cout << key << " was found!" << std::endl;
        found = true;
        return;
    }

    // 2. Recursively search children
    GeneralTree<T>* child = firstChild;
    while (child != nullptr && !found) {
        child->preOrderFind(target, found);
        child = child->nextSibling;
    }
}
```

| | |
|---|---|
| |  |
| Output |  |
| Observation | Calling the function findData on existing nodes such as J outputs that it is found. Otherwise, it outputs 'not found' doesn't if it not found like in the picture above.<br><br>Preorder- this checks the node's own key first. If it matches, it stops early. Otherwise, it recursively searches down through firstChild and sideways through its siblings. (Root -> Left -> Right)<br><br>Postorder - this one is the inverse of pre-order. (Left -> Right-> Root<br><br>Inorder- this one recursively traverses first child subtree then checks the node's own key, and then continues searching through all sibling subtrees (Left -> Root -> Right) |

```
template <typename T>
void GeneralTree<T>::findData(std::string choice, T key) {
    bool found = false;

    if (choice == "PRE")       preOrderFind(key, found);
    else if (choice == "POST") postOrderFind(key, found);
    else if (choice == "IN")   inOrderFind(key, found);
    else {
        std::cout << "Invalid choice.\n";
        return;
    }

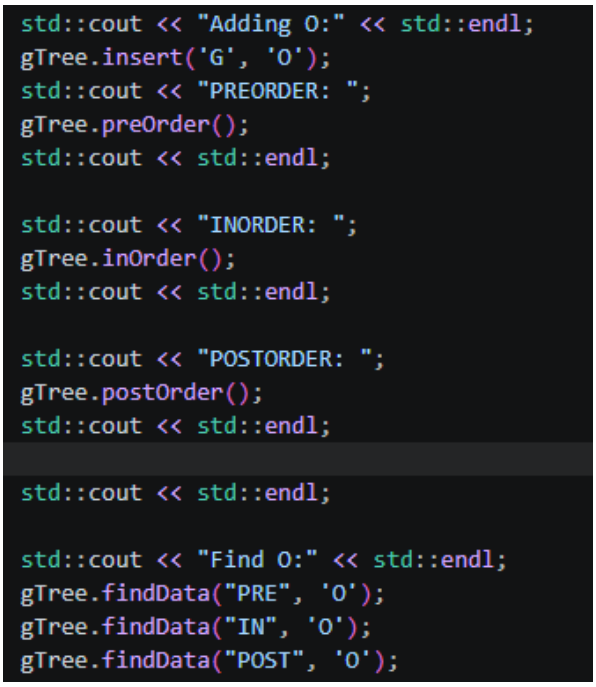
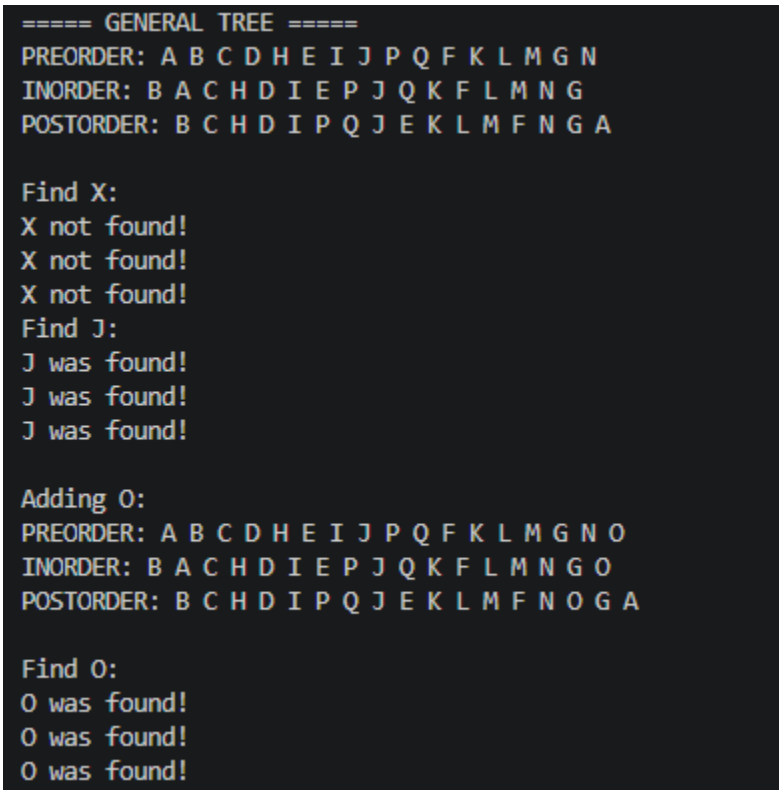
    // Displays message ONLY if entire tree traversal completes without finding the key
    if (!found) {
        std::cout << key << " not found!" << std::endl;
    }
}
```

```
===== GENERAL TREE =====
PREORDER: A B C D H E I J P Q F K L M G N
INORDER: B A C H D I E P J Q K F L M N G
POSTORDER: B C H D I P Q J E K L M F N G A

Find X:
X not found!
Find J:
J was found!
```

Table 9-5. Testing findData function

3.4. Add a new leaf node called which will contain the data value of "O". Make this node the child of node G. Perform the findData function you created in #3, what can you observe about the output? Include your output and answers as table 9-6 in section 6.

| | |
|---|---|
| Code Screenshot |  |
| Output |  |

```
std::cout << "Adding O:" << std::endl;
gTree.insert('G', 'O');
std::cout << "PREORDER: ";
gTree.preOrder();
std::cout << std::endl;

std::cout << "INORDER: ";
gTree.inOrder();
std::cout << std::endl;

std::cout << "POSTORDER: ";
gTree.postOrder();
std::cout << std::endl;

std::cout << std::endl;

std::cout << "Find O:" << std::endl;
gTree.findData("PRE", 'O');
gTree.findData("IN", 'O');
gTree.findData("POST", 'O');
```

```
===== GENERAL TREE =====
PREORDER: A B C D H E I J P Q F K L M G N
INORDER: B A C H D I E P J Q K F L M N G
POSTORDER: B C H D I P Q J E K L M F N G A

Find X:
X not found!
X not found!
X not found!
Find J:
J was found!
J was found!
J was found!

Adding O:
PREORDER: A B C D H E I J P Q F K L M G N O
INORDER: B A C H D I E P J Q K F L M N G O
POSTORDER: B C H D I P Q J E K L M F N O G A

Find O:
O was found!
O was found!
O was found!
```

Table 9-5. Adding leaf node ‘O’ then testing findData

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

**ILO C: Solve given problems using the tree data structure's implementation in C++**

**Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.**

Code Screenshot

```
// create a root node:
BinarySearchTree<int> tree(2);

tree.insert(3);
tree.insert(9);
tree.insert(18);
tree.insert(0);
tree.insert(1);
tree.insert(4);
tree.insert(5);
```

```
template <typename T>
BinarySearchTree<T>::BinarySearchTree(T value){
    key = value;
    left = nullptr;
    right = nullptr;


}
```

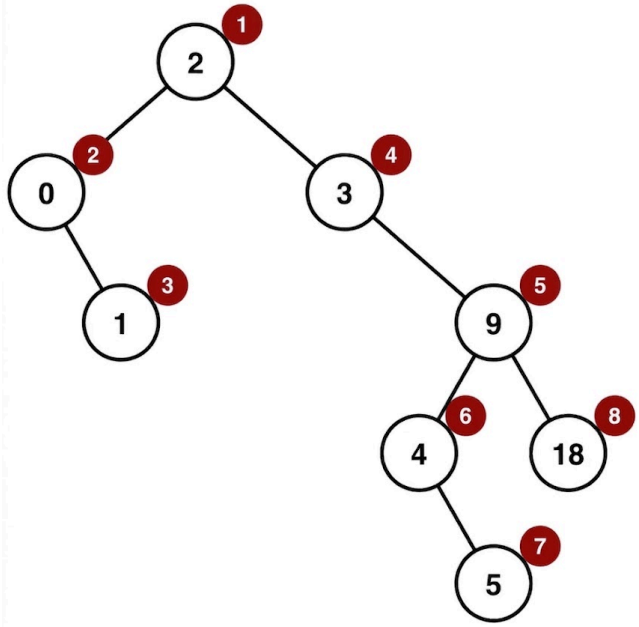
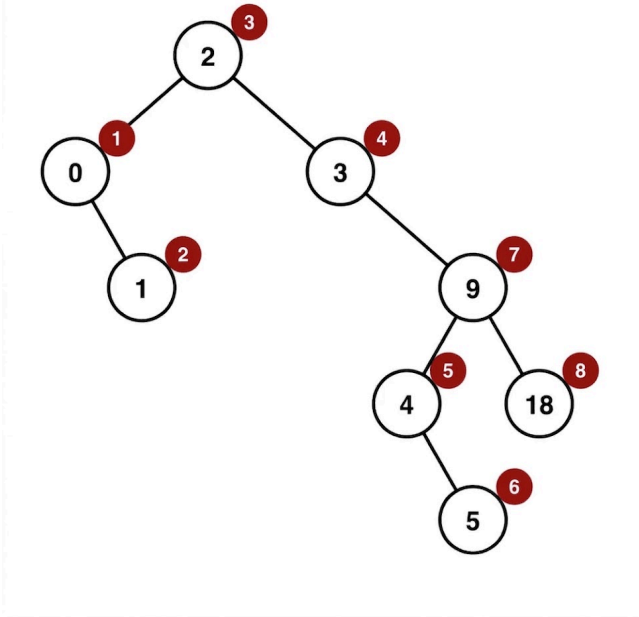
```
template <typename T>
void BinarySearchTree<T>::insert(T value) {
    // left or right

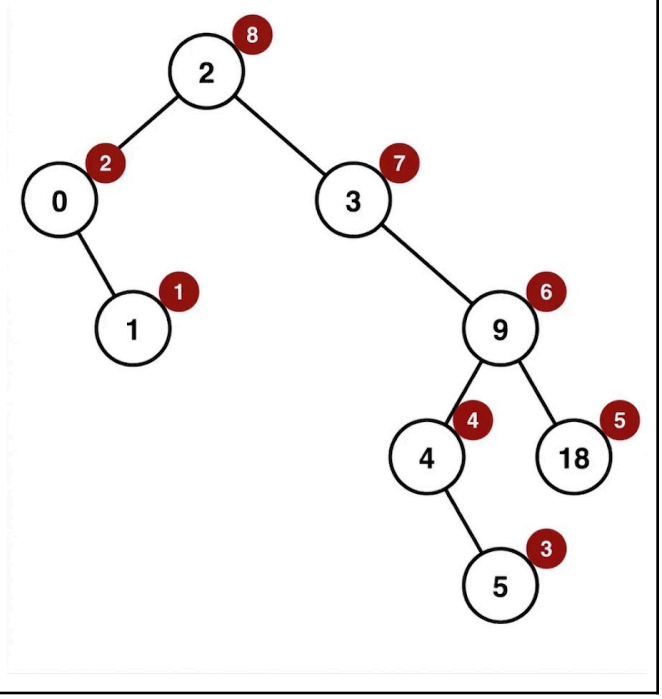

    // go left if val > key // go right if val < key
    if(value < key) {
        if (left == nullptr) {
            left = new BinarySearchTree<T>(value);
        }
        else {
            left->insert(value);
        }
    }
    else if(value > key) {
        if (right == nullptr) {
            right = new BinarySearchTree<T>(value);
        }

        else {
        right->insert(value);
        }

    }
}
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram

| | |
|---|---|
| **PreOrder** |  |
| **InOrder** |  |

| PostOrder |  |
|---|---|



Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

```
PREORDER:
2 0 1 3 9 4 5 18
INORDER:
0 1 2 3 4 5 9 18
POSTORDER:
1 0 5 4 18 9 3 2
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Preorder starts with the root then reads all the values in the left. Afterwards, it goes to the right or in short ROOT->LEFT->RIGHT. In order traverses tree from LEFT -> ROOT -> RIGHT. Lastly. Post order starts reading it from the LEFT -> RIGHT -> ROOT. Looking at the output and the diagram, the logic is the same.

```cpp
template <typename T>
void BinarySearchTree<T>::preOrder() {

    std::cout << key << " ";
    if(left != nullptr) {
        left->preOrder();

    }

    if(right != nullptr) {
        right->preOrder();
    }

}
```

```cpp
template <typename T>
void BinarySearchTree<T>::postOrder() {

    if(left != nullptr) {
        left->postOrder();
    }

    if(right != nullptr) {
        right->postOrder();
    }
    std::cout << key << " ";
}
```

```cpp
template <typename T>
void BinarySearchTree<T>::inOrder() {

    if(left != nullptr) {
        left->inOrder();

    }
    std::cout << key << " ";

    if(right != nullptr) {
        right->inOrder();
    }

}
```

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Code assistance**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **I used AI for some codes since I can't translate to code what I am trying to do like in the linked list implementation.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.